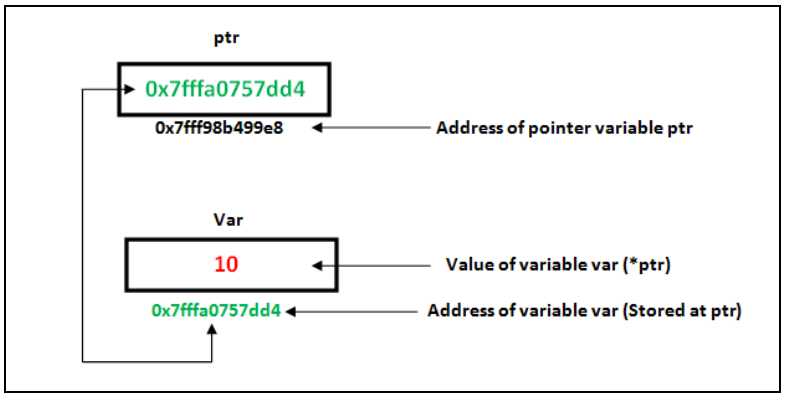
Pointer and Reference Types

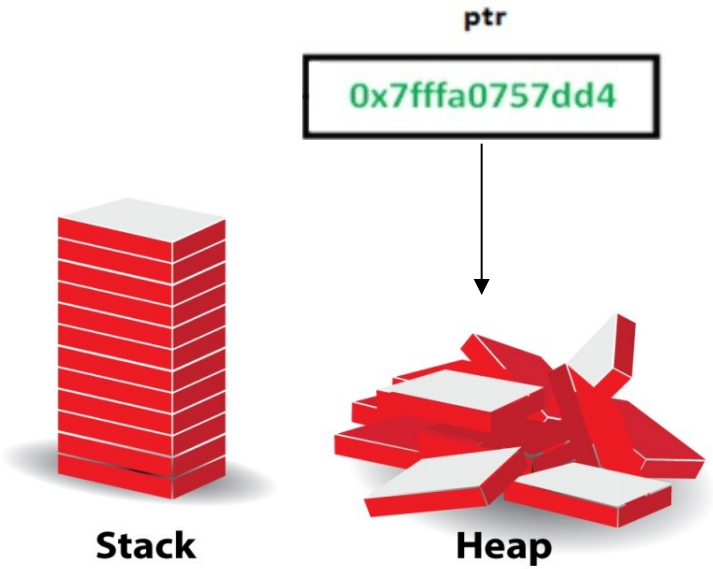


A *pointer* type variable has a range of values that consists of memory addresses and a special value, *nil*

- Provide the power of indirect addressing
- Provide a way to manage dynamic memory
- A pointer can be used to access a location in the area where storage is dynamically created (usually called a heap)

Design Issues of Pointers

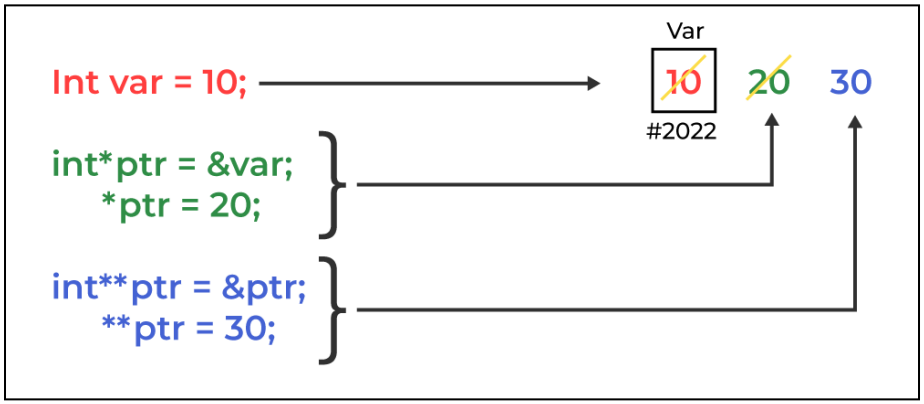
- What are the scope of and lifetime of a pointer variable?
- What is the lifetime of a heap-dynamic variable?
- Are pointers restricted as to the type of value to which they can point?
- Are pointers used for dynamic storage management, indirect addressing, or both?
- Should the language support pointer types?



Pointer Operations

- Two fundamental operations: assignment and dereferencing
- Assignment is used to set a pointer variable's value to some useful address
- Dereferencing yields the value stored at the location represented by the pointer's value

- Dereferencing can be explicit or implicit
- C++ uses an explicit operation via *
- j = *ptr
- sets j to the value located at ptr

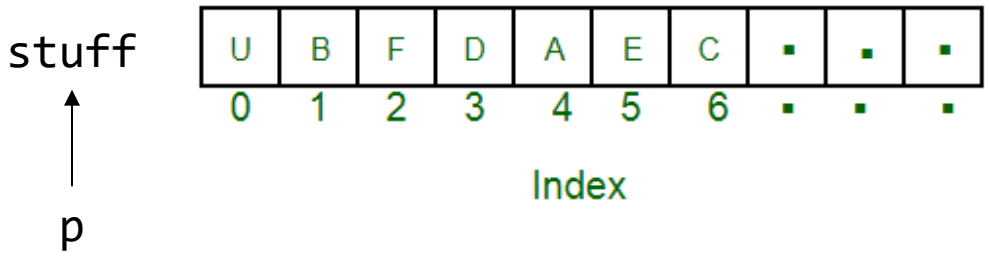


Pointer Arithmetic in C and C++

```
float stuff[100];
float *p;
p = stuff;
```

`*(p+5)` is equivalent to `stuff[5]` and `p[5]`

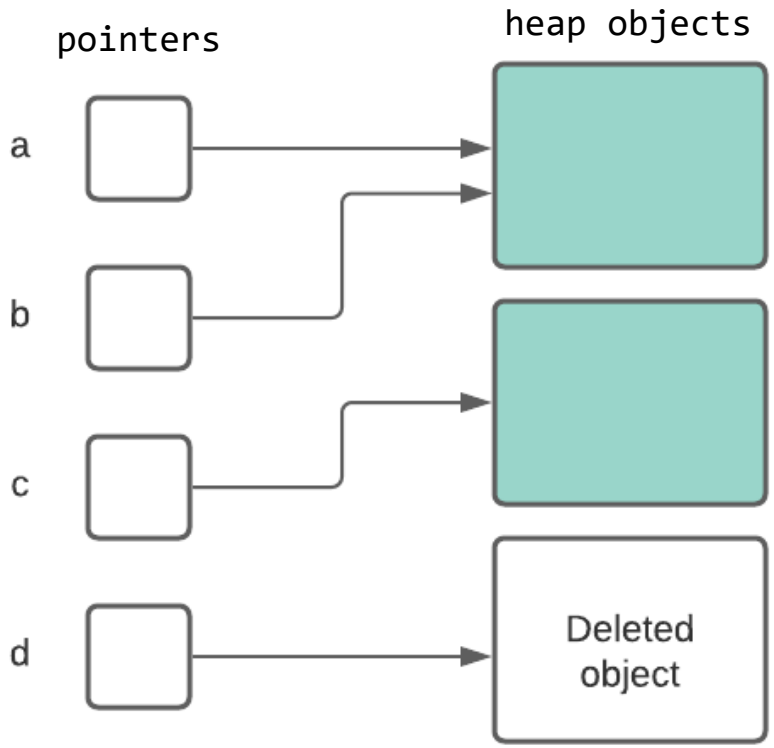
`*(p+i)` is equivalent to `stuff[i]` and `p[i]`



Problems with Pointers

- Dangling pointers (dangerous)

  A pointer points to a heap-dynamic variable that has been deallocated



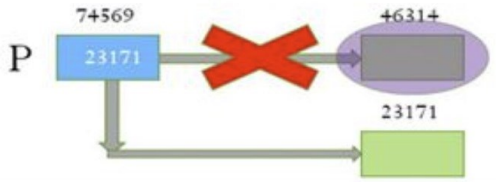
Problems with Pointers

- Lost heap-dynamic variable

  An allocated heap-dynamic variable that is no longer accessible to the user program (often called *garbage*)

  - Pointer `p1` is set to point to a newly created heap-dynamic variable
  - Pointer `p1` is later set to point to another newly created heap-dynamic variable
  - The process of losing heap-dynamic variables is called *memory leakage*

```
int* p;
p= new int;
p= new int;
```

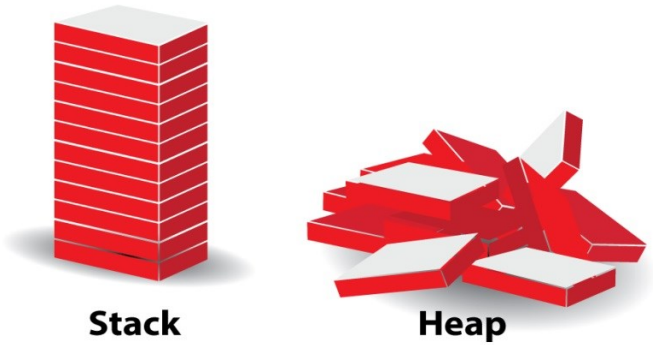


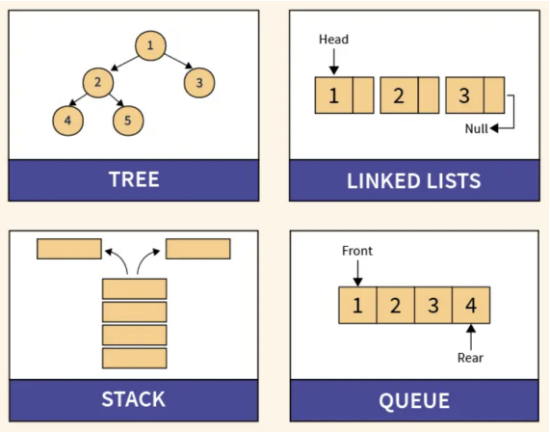
Reference Types

- C++ includes a special kind of pointer type called a *reference type* that is used primarily for formal parameters
  - Advantages of both pass-by-reference and pass-by-value
- Java extends C++'s reference variables and allows them to replace pointers entirely
  - References are references to objects, rather than being addresses
- C# includes both the references of Java and the pointers of C++

Evaluation of Pointers

- Dangling pointers and dangling objects are problems as is heap management
- Pointers are like `goto`'s--they widen the range of cells that can be accessed by a variable





Pointers or references are necessary for dynamic data structures -- so we can't design a language without them

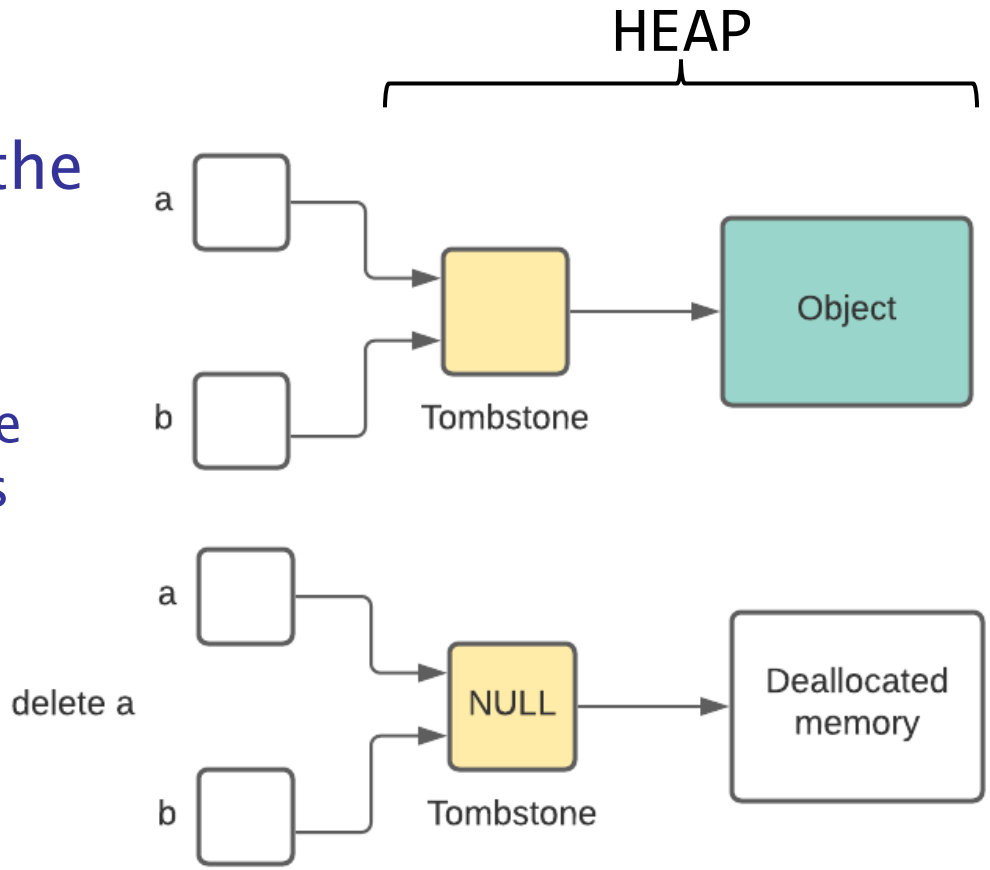
```
goto label;
... .. ...
... .. ...

label:
... .. ...
... .. ...
```

Dangling Pointer Problem

Tombstone: extra heap cell that is a pointer to the heap-dynamic variable

- The actual pointer variable points only at tombstones
- When heap-dynamic variable de-allocated, tombstone remains but set to nil
- Costly in time and space



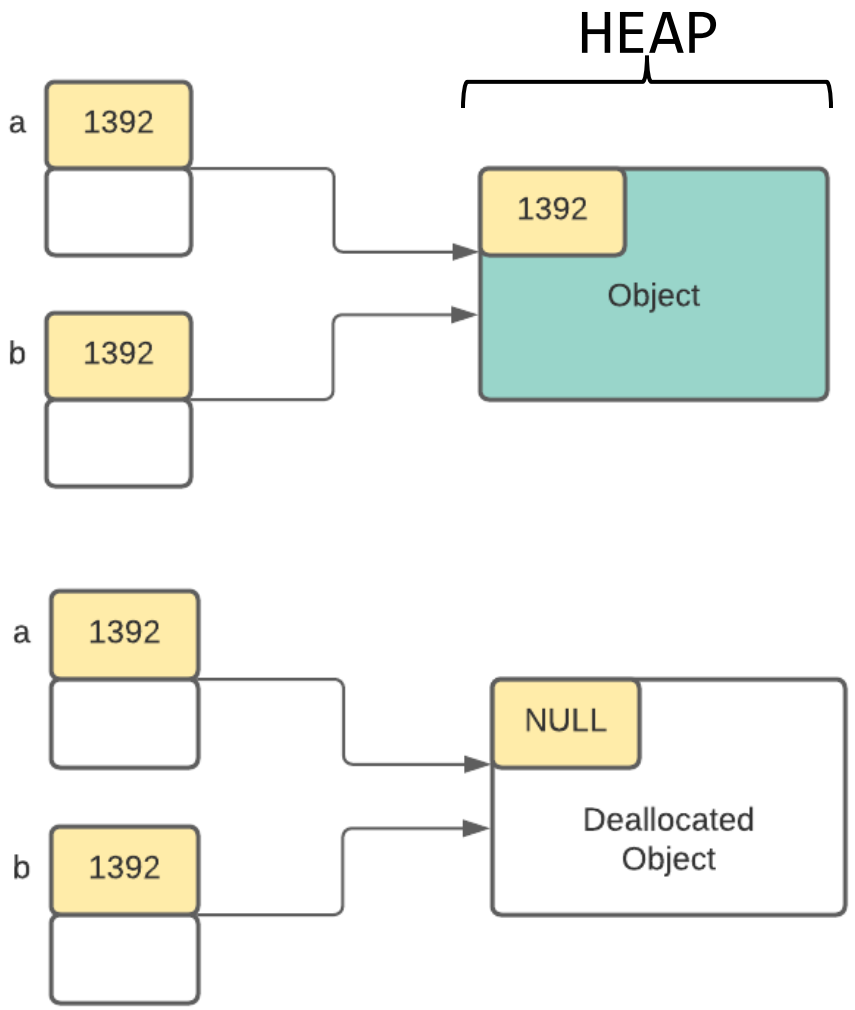

```
int *a = 10
int *b = a

delete a
```

Dangling Pointer Problem

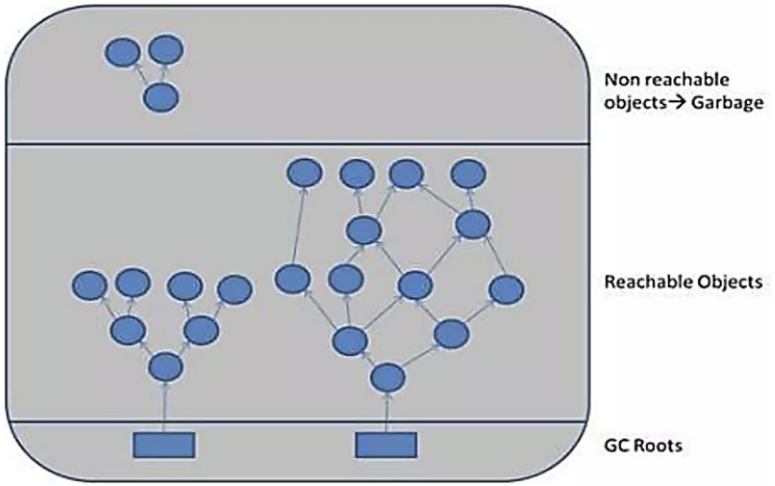
Locks-and-keys: Pointer values are represented as (key, address) pairs

- Heap-dynamic variables are represented as variable plus cell for integer lock value
- When heap-dynamic variable allocated, lock value is created and placed in lock cell and key cell of pointer



Heap Management

- A very complex run-time process
- Single-size cells vs. variable-size cells
- Two approaches to reclaim garbage

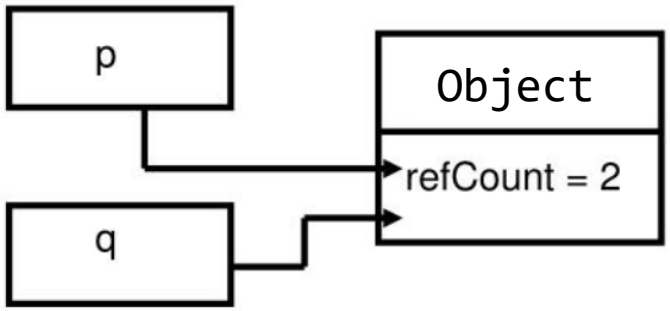


1. Reference counters (eager approach): reclamation is gradual
2. Mark-sweep (lazy approach): reclamation occurs when the list of variable space becomes empty

Reference Counter

Reference counters: maintain a counter in every cell that store the number of pointers currently pointing at the cell

```
Object p= new Integer(57);
Object q = p;
```

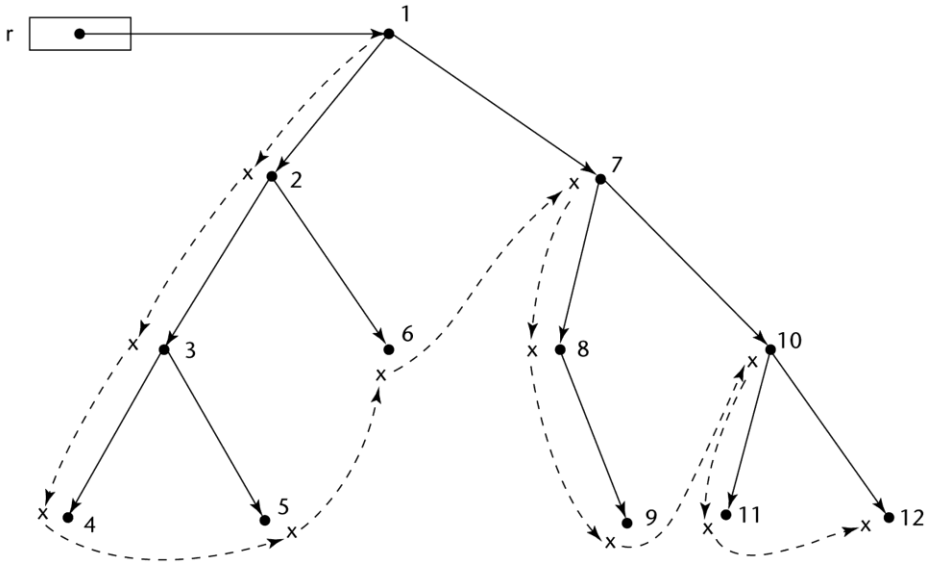


- *Disadvantages*: space required, execution time required, complications for cells connected circularly, heap fragmentation
- *Advantage*: it is intrinsically incremental, easy to implement, no significant delays in the application execution

Mark-Sweep

The run-time system allocates storage cells as requested and disconnects pointers from cells as necessary:

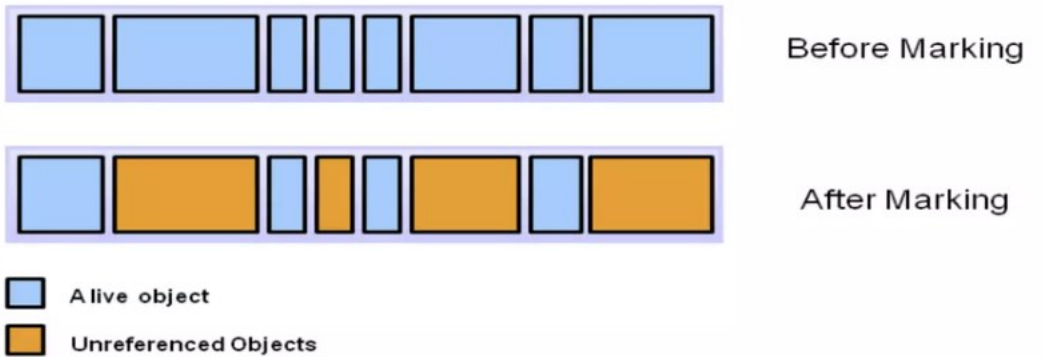
1. Every heap cell has an extra bit used by collection algorithm
2. All cells initially set to garbage
3. All pointers traced into heap, and reachable cells marked as not garbage (mark phase)
4. All garbage cells returned to list of available cells (sweep phase)

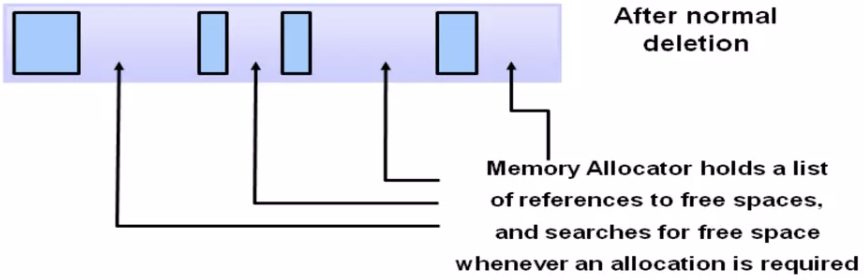


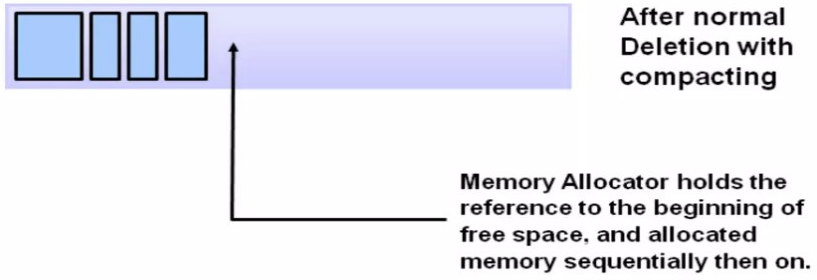
Dashed lines show the order of node_marking

Disadvantages: in its original form, it was done too infrequently. When done, it caused significant delays (halting) in application execution. Contemporary mark-sweep algorithms avoid this by doing it more often—called incremental mark-sweep

Marking Algorithm





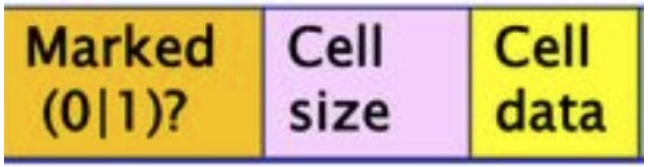


Normal deletion removes unreferenced objects leaving referenced objects and pointers to free space. The memory allocator holds references to blocks of free space where new objects can be allocated

To further improve performance, we can also compact the remaining referenced objects. This makes new memory allocation much easier and faster.

Variable-Size Cells

- All the difficulties of fixed-size cells plus more
- Required by most programming languages
- If mark-sweep is used, additional problems occur
  - The initial setting of the indicators of all cells in the heap is difficult
  - The marking process in nontrivial
  - Maintaining the list of available space is another source of overhead



Rust Smart Pointers

- Box<T> is the simplest smart pointer in Rust, providing heap-allocated storage for a value of type T.
- When a Box<T> goes out of scope, it automatically deallocates the memory it holds.

```
let b = Box::new(5); // Allocates an integer on the heap
// Derefencing happens automatically when b goes out of
scope
```

Rust Smart Pointers

- Rc<T>: when you need multiple parts of your program to own the same data.
- The data will be deallocated only when the last Rc<T> pointing to it goes out of scope.

```
let x = Rc::new(5); // Rc is reference counted
let y = Rc::clone(&x); // Both x and y now share
ownership of the value
```